SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – PROMPT ENGINEERING CHALLENGE

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 1 – Prompt Engineering
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (2 Questions)
CO2 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)*		
Student Name:	ALLAPAREDDY JAHNAVI	**Reg. No.:**	192472214
Section / Batch:	CSE AI/2024	**Date:**	

Instructions to Students
• Answer 2 questionss.Each questions carries 50 marks..Total100 Marks.
1. Design appropriate prompts for the given tasks using suitable prompt engineering techniques.
2. Select and apply the appropriate prompting strategy (Zero-shot, One-shot, Few-shot, Chain-of-Thought, or Structured Prompting) based on the given scenario.
3. Include clear instructions, relevant context, necessary constraints, and the expected output format while designing each prompt.
4. Evaluate and refine the designed prompts to improve their clarity, effectiveness, and quality by applying prompt engineering best practices.
5. Design prompts for a variety of real-world applications, such as text generation, summarization, translation, question answering, code generation, and conversational AI.
6. Justify the designed prompt by briefly explaining the selected prompting technique and how it helps achieve the desired output.

Code of Conduct
I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – PROMPT ENGINEERING CHALLENGE

1. Design a Zero-shot prompt to summarize a news article in 100 words.
2. Create a One-shot prompt to translate an English paragraph into Tamil using one example.
3. Design a Few-shot prompt to classify movie reviews as Positive, Negative, or Neutral.
4. Write a Chain-of-Thought prompt to solve a multi-step mathematical problem.
5. Create a Structured Prompt to generate a professional email requesting leave from work.
6. Design a prompt to generate a Python program that checks whether a number is prime.
7. Write a prompt to generate SQL queries for retrieving student records from a database.
8. Create a prompt to summarize a research paper into key points.

9. Design a prompt to generate interview questions for a Software Engineer position.

10. Write a prompt to explain Artificial Intelligence to a school student using simple language.

11. Design a prompt to create a lesson plan for teaching Machine Learning to undergraduate students.

12. Create a prompt to generate a travel itinerary for a three-day trip to Chennai.

13. Write a prompt for a recipe recommendation system based on available ingredients.

14. Design a prompt for a customer support chatbot to answer product-related queries.

15. Create a prompt to generate social media captions for a new smartphone launch.

16. Write a prompt to generate a product description for an e-commerce website.

17. Design a prompt for performing sentiment analysis on customer feedback.

18. Create a prompt to perform grammar correction on a given paragraph.

19. Write a prompt to convert technical content into simple language for beginners.

20. Design a prompt to generate multiple-choice questions from a textbook chapter.

21. Create a prompt to generate a project abstract from a given project title.

22. Write a prompt to generate personalized study notes for engineering students.

23. Design a prompt to generate a professional resume based on candidate details.

24. Create a prompt to write a cover letter for a Software Developer position.

25. Write a prompt to generate Python code documentation from source code.

26. Design a prompt that explains a programming error and suggests possible fixes.

27. Create a prompt to generate unit test cases for a Python function.

28. Write a prompt to compare BERT, GPT, LLaMA, and Gemini in tabular format.

29. Design a prompt to explain the Transformer architecture with a suitable diagram description.

30. Create a prompt to compare Pre-training and Fine-tuning with suitable examples.

31. Write a prompt to explain the concept of Transfer Learning for beginners.

32. Design a prompt to explain the advantages of LoRA and PEFT in fine-tuning Large Language Models.

33. Create a prompt that demonstrates Function Calling for retrieving weather information.

34. Write a prompt using the OpenAI API to generate a customer support response.

35. Design a prompt using the Gemini API to summarize meeting minutes.

36. Create a prompt using the Hugging Face API to perform sentiment analysis.

37. Write a prompt to evaluate and improve a poorly written text generation prompt.

38. Design a prompt following Prompt Engineering Best Practices to generate educational content.

39. Create a prompt for an AI Career Guidance Assistant that recommends suitable career paths based on student interests.

40. Design a comprehensive prompt for an AI-powered Engineering Assistant that answers technical questions, explains concepts, and provides code examples with appropriate formatting.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technique Selection	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Output Effectiveness	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Prompt Craftsmanship	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Prompt-Output Documentation	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Design Rationale	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Question No.	Technique Selection (30%)	Output Effectiveness (25%)	Prompt Craftsmanship (20%)	Prompt-Output Documentation (15%)	Design Rationale (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

3) **Design a Few-shot Prompt to Classify Movie Reviews as Positive, Negative, or Neutral**

1. Aim

To design, test, evaluate, and refine a few-shot prompt that classifies movie reviews into Positive, Negative, or Neutral sentiment categories.

2. Problem Statement

Movie review platforms receive large amounts of text feedback. Manually reading and categorizing every review is time-consuming. A sentiment classification prompt can help organize reviews according to the opinion they express. The challenge is to distinguish praise, criticism, and factual statements, including reviews that contain both positive and negative comments.

3. Technique Selected: Few-shot Prompting

Few-shot prompting provides the model with several input-output examples before asking it to handle a new case. The examples demonstrate the expected relationship between a review and its label. It is suitable for this task because three labels must be distinguished consistently.

Technique	Description	Suitability for this task
Zero-shot	Gives instructions without examples.	May work, but gives no sample label pattern.
One-shot	Provides one labelled example.	Shows a pattern but may not demonstrate all categories.
Few-shot (selected)	Provides examples for multiple categories.	Demonstrates Positive, Negative, and Neutral labels.
Structured prompting	Organizes instructions into sections.	Can support the prompt, but examples are the main technique here.

4. Designed Prompt

The following prompt is the complete prompt to give to a Generative AI model:

ROLE: You are a movie-review sentiment classification assistant.

TASK: Classify the supplied review into exactly one category: Positive, Negative, or Neutral.

DEFINITIONS:

• Positive: the overall opinion expresses satisfaction, praise, or recommendation.

• Negative: the overall opinion expresses dissatisfaction, criticism, or disappointment.

• Neutral: the review is mainly factual or does not express a clear positive or negative opinion.

INSTRUCTIONS:

1. Read the entire review before deciding.

2. Consider the overall meaning, not just one positive or negative word.

3. For a mixed review, choose the dominant overall sentiment.

4. Do not invent details or add a fourth category.

5. Return only the required label.

EXAMPLES:

Review: "The movie was fantastic, and the actors delivered outstanding performances."

Sentiment: Positive

Review: "The story was boring, the acting was poor, and the movie was a disappointment."

Sentiment: Negative

Review: "The film was released last Friday and has a running time of two hours."

Sentiment: Neutral

NEW REVIEW: The movie had beautiful visuals, but the slow story made it difficult to enjoy.

OUTPUT FORMAT: Sentiment: [Positive / Negative / Neutral]

5. Sample Input and Expected Output

Item	**Content**
Input review	The movie had beautiful visuals, but the slow story made it difficult to enjoy.
Expected label	Negative
Explanation for evaluation	The review praises the visuals, but its overall judgment is unfavorable because the slow story reduced enjoyment.

6. Workflow / Flowchart

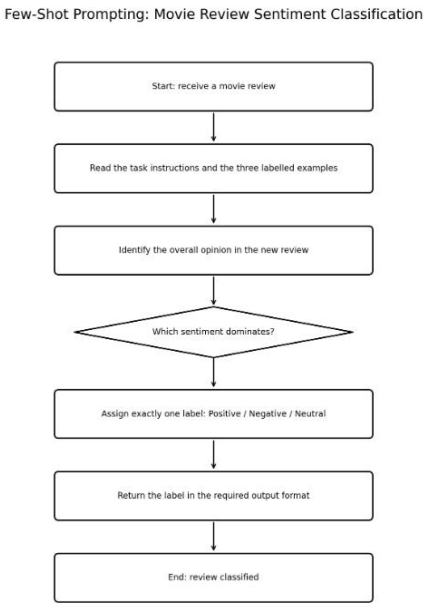


Figure 2. Workflow for few-shot movie-review sentiment classification.

The flowchart shows how the task instructions and labelled examples guide the classification of a new

review. The final response is restricted to one of the three permitted labels.

7. Classification Examples and Test Table

No.	Test review	Expected class	Reason
1	The performances were excellent and the story was moving.	Positive	Clear praise.
2	The plot was confusing and I regretted watching it.	Negative	Clear dissatisfaction.
3	The movie premiered in 2024 and lasts 110 minutes.	Neutral	Factual information only.
4	The visuals were lovely, but the story was dull.	Negative	Overall opinion is unfavorable.
5	A well-made film with memorable performances.	Positive	Praise dominates.

8. Evaluation of Prompt Quality

Criterion	How it is addressed	Expected benefit
Technique selection	Examples cover all three sentiment classes.	Shows the model the desired mapping.
Clarity	Role, task, definitions, and rules are explicit.	Reduces ambiguity.
Output effectiveness	Only one of three labels is allowed.	Makes output consistent and easy to process.
Context	Examples are short and relevant to movie reviews.	Keeps the task focused.
Robustness	Mixed-review rule is included.	Encourages consideration of the whole review.
Limitations	Sarcasm and subtle context may remain difficult.	Identifies cases needing human review.

9. Prompt Refinement

A basic prompt such as “Classify this movie review” is underspecified. It does not define the categories, show examples, or constrain the output. The refined prompt improves the design in the following ways:

Adds clear definitions for Positive, Negative, and Neutral.

Includes one example for each category so the model can compare patterns.

Instructs the model to consider the full review rather than isolated words.

Defines how to handle mixed sentiment by selecting the dominant overall opinion.

Specifies an exact output format and prohibits extra categories.

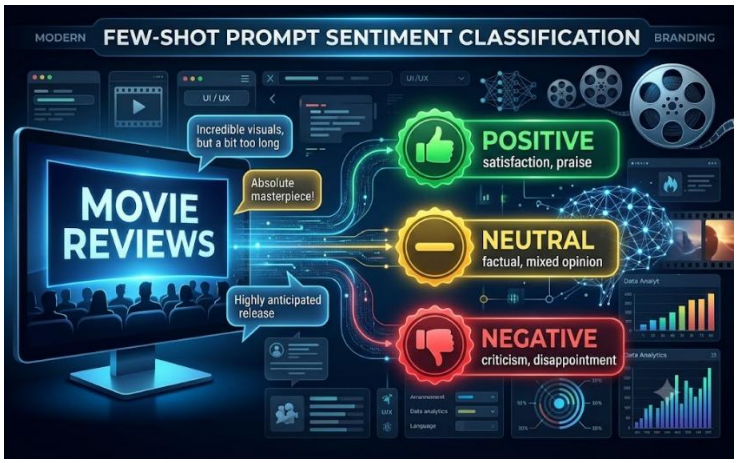
Refinement should be validated by testing the prompt on reviews with clear sentiment, mixed sentiment, factual language, and sarcasm. If errors occur, add representative examples and revise the category rules.

10. Applications, Advantages, and Limitations

Applications	Advantages	Limitations
Movie review websites	Fast and consistent initial categorization.	Sarcasm may be misinterpreted.
Customer feedback analysis	Can process many short comments.	Quality depends on the input and model.
Social media monitoring	Supports trend analysis across opinions.	Context and slang may be ambiguous.
Market research	Helps summarize audience reactions.	Human review may be needed for high-impact decisions.

11. Design Rationale

Few-shot prompting is the best primary choice because the model needs to learn the expected relationship between review wording and three possible labels from examples in the prompt. The examples cover each class, while definitions and output constraints reduce ambiguity. The prompt is not guaranteed to be correct for every review, so testing and refinement remain important.



12. Result

A few-shot prompt was designed for movie-review sentiment classification. It contains clear category definitions, three labelled examples, instructions for mixed sentiment, and a restricted output format. The sample review is classified as Negative because its overall opinion is unfavorable. The prompt is suitable for an introductory sentiment-analysis workflow.

6) Design a Structured Prompt to Generate a Python Program That Checks Whether a Number Is Prime

1. Aim

To design, test, evaluate, and refine a structured prompt that generates a correct, readable, and efficient Python program for prime-number checking.

2. Problem Statement

A prime number is an integer greater than 1 with exactly two positive divisors: 1 and itself. A code-generation prompt must state this definition clearly and account for boundary cases such as negative integers, 0, 1, 2, and composite numbers. It should also request readable code, an explanation, and test outputs.

3. Technique Selected: Structured Prompting

Structured prompting organizes a request into named sections such as role, task, definition, requirements, constraints, and output format. It is appropriate for code generation because missing or vague requirements can lead to incorrect logic or incomplete answers.

Prompt component	**Purpose**	**Example in this task**
Role	Sets the expected expertise.	Act as a Python programming instructor.
Task	States what must be produced.	Write a prime-number checker.
Context / definition	Defines the concept accurately.	A prime number is greater than 1 with exactly two positive divisors.
Requirements	Lists functional behavior.	Read input, test divisors, print result.
Constraints	Limits implementation choices.	Python 3, no external libraries.
Output format	Specifies response organization.	Algorithm, code, sample inputs/outputs, result.

4. Designed Prompt

The following structured prompt can be given to a Generative AI model:

ROLE: Act as an experienced Python programmer and programming instructor.

TASK: Generate a Python program that determines whether an integer is prime.

DEFINITION: A prime number is an integer greater than 1 that has exactly two positive divisors: 1 and itself.

FUNCTIONAL REQUIREMENTS:

1. Read one integer from the user.
2. Display whether it is prime or not prime.
3. Treat all integers less than or equal to 1 as not prime.

4. Handle 2 correctly as prime.

5. Check possible divisors only up to the integer square root of the number.

6. Stop checking as soon as a divisor is found.

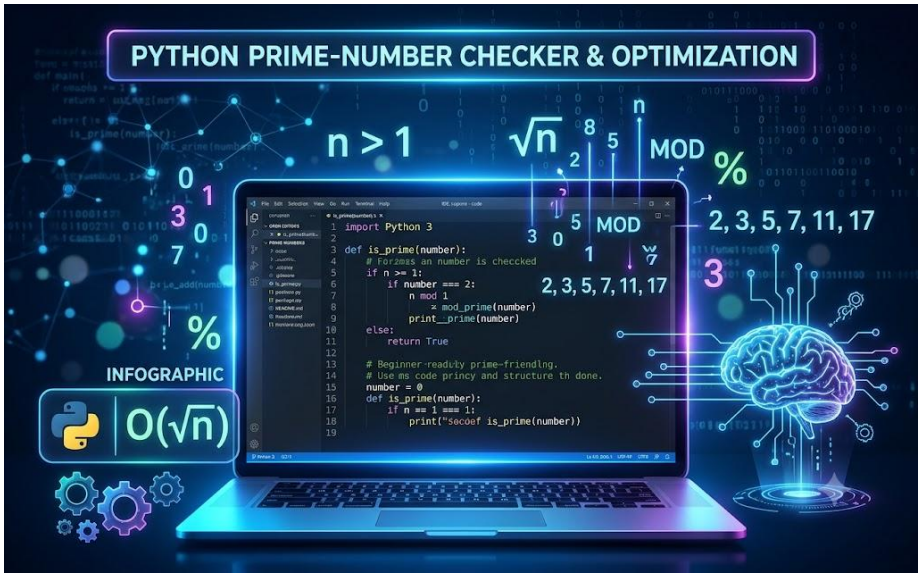
CONSTRAINTS:

• Use Python 3 syntax.

• Do not use external libraries.

• Use meaningful variable names and beginner-friendly logic.

• Avoid unnecessary complexity.



EXPECTED RESPONSE FORMAT:

A. Brief algorithm or pseudocode.

B. Complete executable Python source code.

C. Sample output for a prime number and a non-prime number.

D. Test cases for -5, 0, 1, 2, 9, and 17.

E. A short explanation of the logic and time complexity.

QUALITY CHECK: Verify the boundary cases and ensure the code follows the definition before presenting the final answer.

5. Algorithm / Pseudocode

START

INPUT n

IF n <= 1

OUTPUT "Not prime"

ELSE

SET is_prime = True

FOR divisor FROM 2 TO floor(sqrt(n))

IF n MOD divisor == 0

SET is_prime = False

STOP LOOP

IF is_prime == True

OUTPUT "Prime"

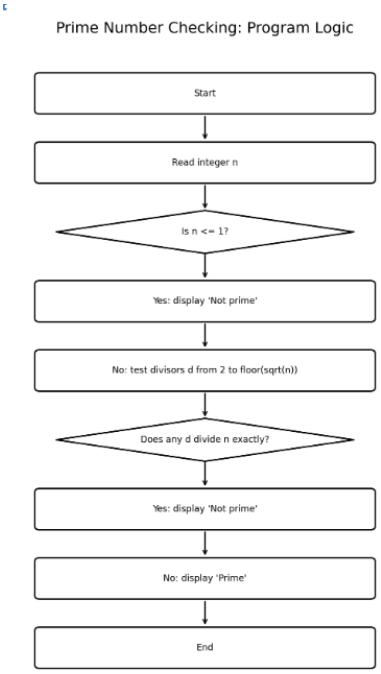
ELSE

OUTPUT "Not prime"

END IF

END

6. Flowchart: Program logic for checking whether an integer is prime.



7. Generated Python Program

number = int(input("Enter an integer: "))

if number <= 1:

print(number, "is not a prime number.")

else:

is_prime = True

for divisor in range(2, int(number ** 0.5) + 1):

if number % divisor == 0:

```
is_prime = False
break


if is_prime:
print(number, "is a prime number.")
else:
print(number, "is not a prime number.")
```

8. Code Explanation

Code element	Explanation
input() and int()	Accept the user's input and convert it to an integer.
number <= 1	Correctly excludes negative integers, 0, and 1 from the prime category.
is_prime = True	Initially assumes the number is prime.
range(2, int(number ** 0.5) + 1)	Checks possible divisors from 2 through the integer square root.
number % divisor == 0	Tests whether the number is exactly divisible by a candidate divisor.
break	Stops the loop once a divisor is found.
Final if-else	Prints the appropriate result based on the flag.

9. Testing and Expected Outputs

Input	Expected output	Reason
-5	-5 is not a prime number.	Negative integer.
0	0 is not a prime number.	Zero is not prime.
1	1 is not a prime number.	Prime numbers must be greater than 1.
2	2 is a prime number.	The smallest prime number.
9	9 is not a prime number.	Divisible by 3.
17	17 is a prime number.	No divisor from 2 through floor(sqrt(17)).

Sample run 1:

Enter an integer: 7

7 is a prime number.

Sample run 2:

Enter an integer: 12

12 is not a prime number.

10. Evaluation of Prompt Quality

Criterion	Evidence in the prompt	Benefit
Technique selection	Requirements are grouped into named sections.	Reduces missing requirements.
Correctness	Mathematical definition and boundary cases are explicit.	Prevents common logical errors.
Efficiency	Requests divisor checking up to square root only.	Avoids unnecessary iterations.
Output effectiveness	Requests algorithm, code, tests, and explanation.	Makes the answer complete and verifiable.
Readability	Requires meaningful names and beginner-friendly code.	Helps students understand the solution.
Verification	Includes a quality-check instruction and test cases.	Encourages systematic validation.

11. Prompt Refinement

A weak prompt such as "Write a program to check prime numbers" does not define prime numbers, specify a language version, identify boundary cases, or request tests. The refined prompt adds these missing details.

Defines prime numbers mathematically.

Specifies Python 3 and prohibits external libraries.

Requires correct handling of negative values, 0, 1, and 2.

Requests the square-root optimization and early termination after finding a divisor.

Requires a clear output format, code explanation, and test cases.

Adds a verification instruction so the generated solution is checked against the stated requirements.

The code should then be executed with the test inputs listed above. If any output differs from the expected result, the prompt should be revised or the code corrected before submission.

12. Efficiency Analysis

Method	Maximum divisor checks	Time complexity	Comment
Check all integers from 2 to n-1	Approximately n-2	O(n)	Simple but slower for large n.
Check divisors up to sqrt(n)	Approximately sqrt(n)-1	O(√n)	More efficient for this basic primality test.

The square-root method works because if n = a × b and both a and b were greater than √n, their product would be greater than n. Therefore, a composite number must have at least one factor no greater than √n.

13. Design Rationale, Applications, and Limitations

Structured prompting was selected because a programming task has several independent requirements: correct logic, boundary handling, efficiency, readable code, and test evidence. Named sections make each requirement explicit and make the resulting response easier to evaluate.

Applications	**Advantages**	**Limitations**
Programming education	Produces code with explanations and tests.	Generated code still needs execution.
Coding practice tools	Makes requirements and output consistent.	Unclear prompts can still produce errors.
Algorithm demonstrations	Connects mathematical logic to implementation.	The program expects valid integer input.
Rapid prototyping	Speeds up the first draft of simple code.	Human review is needed before real deployment.

14. Result

A structured prompt was developed to generate a Python 3 program for prime-number checking. The prompt specifies the mathematical definition, functional requirements, boundary cases, square-root optimization, output format, and validation examples. The resulting program correctly handles the listed test cases and has O(√n) time complexity for the divisor-checking loop.